Лабораторная работа №3. Метрики программного кода

Дисциплина: Технологии разработки программного обеспечения

Проект: Investment Assistant — Telegram-бот, инвестиционный ассистент на T-Invest MCP (Python 3.12)

Репозиторий: https://github.com/Rudimix/InvestmentAssistant, ветка `feature/assistant-and-lab3-metrics`

1. Задание

1. Построить не менее 5 метрик программного кода.
2. Проанализировать результаты.
3. Оптимизировать код на основе метрик.
4. Перестроить метрики для улучшенного кода.
5. Сделать выводы.

В методичке метрики строятся средствами Visual Studio для C# (меню «Анализ → Метрики кода», `msbuild /t:Metrics`, правила CA1501/CA1502/CA1505/CA1506). Проект написан на Python, поэтому используются готовые инструменты анализа Python-кода, которые считают те же метрики.

2. Объект измерения

Пакет `src/investment_journal` (12 модулей, около 1200 строк):

Модуль	Назначение
`models`	Модели: пользователь, черновик и решение журнала, свеча, позиция, портфель
`config`	Настройки из переменных окружения и `.env`
`mcp_client`	Клиент MCP-сервера T-Invest (stdio), разбор ответов инструментов
`market`	Цена, свечи, портфель через MCP-инструменты `get_last_price`, `get_candles`, `get_portfolio`
`storage`	Хранилище журнала в SQLite
`journal`	Сценарии: записать гипотезу с текущей ценой, оценить, закрыть
`analytics`	Оценка гипотезы (buy/sell/hold), статистика свечей: SMA, волатильность, просадка, тренд
`formatting`	Тексты сообщений бота
`llm`	Свободный диалог: любая LLM с OpenAI-совместимым API вызывает MCP-инструменты (только чтение)
`bot`	Обработчики Telegram (aiogram 3): `/price`, `/stats`, `/portfolio`, `/idea`, `/journal`, `/review`, `/close`
`__main__`	Сборка приложения и запуск

3. Инструменты

Метрика Visual Studio	Инструмент Python	Команда	Комментарий
Maintainability Index	**radon**	`radon mi -s`	Шкала 0–100. Ранги A (> 19), B (10–19), C (< 10) совпадают с зелёной, жёлтой и красной зонами VS. Считается для модуля целиком
Cyclomatic Complexity	**radon**	`radon cc -s -a`	По функциям и методам, ранги A–F. Для перекрёстной проверки — **lizard** (CCN) и **pyscn**
Правило CA1502 (CC > N)	**flake8** + **mccabe**	`flake8 --select C901`	Порог `max-complexity = 10` в `.flake8`, как в примере методички. Дополнительно `lizard -C 10 -w`
Class Coupling	**pyscn**	`pyscn analyze --json`	CBO (Coupling Between Objects) по классам. Связи модулей: Ca, Ce и нестабильность I = Ce / (Ca + Ce)
Depth of Inheritance	**pylint**	`pylint --enable=too-many-ancestors --max-parents=0`	R0901 выводит число предков для каждого класса. pylint не считает `object`, поэтому DIT VS = значение pylint + 1
Правила CA15xx	**pylint** (design checker)	`pylint --enable=design`	Пороги в `[tool.pylint.design]` в `pyproject.toml`: `max-parents = 5` (≈ CA1501), `max-branches`, `max-locals` и др.
Source Lines / Executable Lines	**radon**	`radon raw -s`	LOC — все строки, LLOC — логические строки (операторы), SLOC, комментарии, пустые
—	**radon**	`radon hal`	Метрики Холстеда: объём V, сложность, оценка числа ошибок B
—	**pyscn**	`pyscn analyze`	Когнитивная сложность, LCOM4, дублирование кода, мёртвый код, итоговая оценка health score
—	**coverage.py**	`coverage run -m unittest`	Покрытие операторов и ветвей тестами

Все команды собраны в `scripts/metrics.ps1`. Скрипт только запускает инструменты и сохраняет их вывод без изменений:

```
python -m pip install -e ".[metrics]"
.\scripts\metrics.ps1 -Out metrics\after
```

Отчёты: `metrics/before/` (до оптимизации) и `metrics/after/` (после). HTML-отчёт pyscn — `pyscn.html` в каждом каталоге.

Замер «до» снят с коммита `f91db41` (первая версия), замер «после» — с итогового коммита ветки.

4. Метрики исходного кода (до оптимизации)

4.1. Сводка

Метрика	Инструмент	Значение
Функций и методов	radon cc	66
Цикломатическая сложность: средняя / максимальная / сумма	radon cc	3,70 / **30** / 244

Метрика	Инструмент	Значение
Функций с рангом CC: A / B / C / D	radon cc	55 / 5 / 5 / **1**
Функций с CC > 10	lizard `-C 10`	**6**
Нарушений C901 (CC > 10)	flake8 + mccabe	**3**
Когнитивная сложность: средняя / максимальная	pyscn	2,77 / **38**
Средний MI модулей (без `__init__`)	radon mi	48,3 (все модули — ранг A)
CBO: средний / максимальный	pyscn	0,64 / 4
LCOM4 (максимум)	pyscn	1
DIT (максимум, по pylint)	pylint R0901	3 (DIT VS = 4)
LOC / SLOC / LLOC	radon raw	1192 / 922 / 853
Дублирование кода / мёртвый код	pyscn	0 % / 0
Оценка pylint	pylint	8,89 / 10
Health score	pyscn	87 (B)
Покрытие (операторы + ветви)	coverage.py	95 %

4.2. Модули

Модуль	MI (radon)	Холстед V	Холстед B	Ca	Ce	I
`__main__`	61,18	4,8	0,002	0	7	1,00
`analytics`	46,25	**818,9**	**0,273**	3	1	0,25
`bot`	**31,46**	282,1	0,094	1	7	0,88
`config`	51,19	215,4	0,072	4	0	0,00
`formatting`	50,26	43,2	0,014	2	2	0,50
`journal`	47,04	97,7	0,033	3	4	0,57
`llm`	35,39	259,0	0,086	2	4	0,67
`market`	**35,19**	**705,3**	**0,235**	3	3	0,50
`mcp_client`	47,77	218,8	0,073	4	1	0,20
`models`	75,96	25,3	0,008	6	0	0,00
`storage`	49,26	53,8	0,018	2	1	0,33

Ca — сколько модулей пакета зависят от модуля, Ce — от скольких модулей пакета зависит он сам (pyscn).

4.3. Самые сложные методы

Метод	CC radon	CCN lizard	CC pyscn	Когнитивная (pyscn)	Вложенность	Строк
`market.MarketDataService.portfolio`	**30** (D)	30	8	**38**	2	46
`analytics.candle_stats`	**18** (C)	18	16	20	3	43
`analytics.review_decision`	**14** (C)	14	14	**35**	3	36
`llm.InvestmentAgent.ask`	**12** (C)	12	9	20	**4**	36
`mcp_client.parse_tool_result`	**12** (C)	12	8	9	3	17

Метод	CC radon	CCN lizard	CC pyscn	Когнитивная (pyscn)	Вложенность	Строк
`config.load_settings`	**11** (C)	11	10	6	1	34
`bot.cmd_stats`	10 (B)	10	8	11	2	26

4.4. Нарушения правил

Инструмент	Сообщение
flake8 C901	`analytics.py:44` `review_decision` is too complex (12)
flake8 C901	`analytics.py:88` `candle_stats` is too complex (11)
flake8 C901	`llm.py:109` `InvestmentAgent.ask` is too complex (11)
lizard `-C 10`	ещё `portfolio` (30), `parse_tool_result` (12), `load_settings` (11)
pylint R0912	`analytics.py:44` Too many branches (16/12) — `review_decision`
pylint R0914	`analytics.py:88` Too many local variables (16/15) — `candle_stats`
pylint R1731	`analytics.py` consider-using-max-builtin (2 раза) — `candle_stats`
pylint W0212	`llm.py` protected-access — обращение агента к `journal.market._tools`
pylint R0902	`CandleStats`, `Position` — 9 полей при пороге 7

5. Анализ результатов

1. **В целом код сопровождаемый.** Все модули имеют ранг A по MI (radon), дублирования и мёртвого кода нет, LCOM4 = 1 у всех классов: каждый класс — одна связная группа методов. Проблемы сосредоточены в шести методах с CC > 10.
2. **`MarketDataService.portfolio` — главная точка риска.** Это единственный метод с рангом D (CC 30) и максимальной когнитивной сложностью (38). Метод четыре раза повторяет одно и то же ветвление «словарь `{units, nano}` или число». Высокое значение подтверждает и метрика Холстеда: у `market` объём 705 и оценка ошибок 0,235 при всего 96 строках, это второе место в пакете. Модуль `market` имеет и один из худших MI — 35,19.
3. **`analytics`** — максимальный объём Холстеда (819). В `review_decision` три почти одинаковые ветки buy/sell/hold с вложенными `if` (когнитивная сложность 35, pylint R0912 — 16 ветвей). `candle_stats` смешивает пять расчётов в одном теле (CC 18, R0914 — 16 локальных переменных, R1731 — ручная реализация `max`).
4. **`InvestmentAgent.ask`** имеет наибольшую вложенность (4). Он совмещает цикл диалога, разбор аргументов от LLM, маршрутизацию по инструментам и обработку ошибок. pylint W0212 указывает на обращение к закрытому полю чужого объекта (`journal.market._tools`) — скрытую связь в обход публичного интерфейса.
5. **`parse_tool_result`, `load_settings`, `cmd_stats`** — сложность на границе порога из-за длинных цепочек проверок.
6. **Инструменты по-разному считают цикломатическую сложность.** У `portfolio` radon и lizard дают 30, а mccabe (flake8) и pyscn — 8: первые две считают каждое `or`/`and` отдельной ветвью (`x.get("units", 0) or 0`), вторые — нет. Поэтому flake8 этот метод не

пометил. Когнитивная сложность (pyscn, 38) подтверждает, что метод действительно трудно читать.

7. **Связанность и наследование в норме.** Максимальный CBO — 4 (`MarketDataService`, `JournalRepository`). Наибольшая эфферентная связанность у слоёв-«склеек» `bot` и `__main__` (Ce = 7, I ≈ 0,9–1), что ожидаемо для внешнего слоя. Самые устойчивые модули — `models` и `config` (I = 0): от них зависят, сами они ни от чего не зависят. Циклических зависимостей нет. Глубина наследования 3 (по pylint) есть только у исключений (`ToolError → RuntimeError → Exception → BaseException`), правило `max-parents = 5` не нарушено.
8. **Покрытие тестами — 95 %.** Модуль `__main__` не покрыт (0 %), потому что сборка приложения смешана с запуском polling.

6. Оптимизация кода на основе метрик

Проблема по метрикам	Что сделано
`portfolio`: CC 30 (D), когнитивная 38, 4 копии ветвления	Выделены `quotation()` (число T-Invest API → `float`), `parse_position()`, `parse_candle()`. Фильтр нулевых валют стал свойством `Position.is_empty_cash`
`review_decision`: CC 14, когнитивная 35, R0912	Табличная диспетчеризация: buy и sell — одна функция `_judge_direction(sign)` со знаком ±1, hold — `_judge_hold`. Словарь `_JUDGES` заменил цепочку `if/elif`
`candle_stats`: CC 18, R0914, R1731	Выделены чистые функции `returns`, `volatility_pct`, `max_drawdown_pct` (со встроенным `max`), `trend`
`InvestmentAgent.ask`: CC 12, вложенность 4, W0212	Выделены `_run_tool`, `_dispatch`, `_remember`, `parse_arguments`, `as_function`. Вместо обращения к закрытому полю добавлен публичный `MarketDataService.list_tools()`
`parse_tool_result`: CC 12	Выделены `_from_text` и `_unwrap`
`load_settings`: CC 11	Выделены `mcp_settings`, `llm_settings`, `split_list`. `read_dotenv` разбит на `_parse_line` и `_unquote`
`cmd_stats`: CC 10	Разбор аргументов вынесен в чистую функцию `parse_stats_args`
`__main__`: покрытие 0 %	Выделена `build_dispatcher()`. Тест на неё сразу показал дефект: модульный `Router` aiogram можно подключить только к одному `Dispatcher`. Регистрация обработчиков переделана на фабрику `create_router()`

Поведение не изменилось: 75 тестов первой версии проходят, их проверки не менялись (в сквозном тесте Telegram только подготовка переведена на `build_dispatcher`). Добавлено 5 тестов на новые функции, итого 80.

7. Метрики после оптимизации

7.1. Сводка

Метрика	Инструмент	До	После
Функций и методов	radon cc	66	92
Средняя CC	radon cc	3,70	**2,63**
Максимальная CC	radon cc	30 (D)	**7 (B)**

Метрика	Инструмент	До	После
Сумма CC	radon cc	244	242
Ранги CC: A / B / C / D	radon cc	55 / 5 / 5 / 1	**85 / 7 / 0 / 0**
Функций с CC > 10	lizard `-C 10`	6	**0**
Нарушений C901	flake8 + mccabe	3	**0**
Средняя CCN / средний NLOC функции	lizard	3,8 / 10,1	**2,7 / 6,7**
Когнитивная сложность: средняя / максимальная	pyscn	2,77 / 38	**1,24 / 7**
Средняя вложенность	pyscn	0,76	0,65
Средний MI модулей	radon mi	48,3	48,2
CBO: средний / максимальный	pyscn	0,64 / 4	0,55 / 4
LCOM4 (максимум)	pyscn	1	1
DIT (максимум, pylint)	pylint	3	3
LOC / SLOC / LLOC	radon raw	1192 / 922 / 853	1240 / 898 / 847
Сообщений pylint design (R09xx)	pylint	5	3
Оценка pylint	pylint	8,89	8,98
Health score	pyscn	87 (B)	88 (B), complexity score 95 → 100
Покрытие	coverage.py	95 %	**97 %**

Остались три сообщения pylint design: R0902 у dataclass-ов `CandleStats` и `Position` (9 полей — это состав данных, а не логика) и R0903 у `IdeaForm` (группа состояний aiogram, методы ей не нужны). Все три — осознанные исключения.

7.2. Методы, нарушавшие правила

Метод	CC radon	CC pyscn	Когнитивная	Строк (pyscn)
`market.MarketDataService.portfolio`	30 → **7**	8 → 4	38 → **3**	46 → 9
`analytics.candle_stats`	18 → **6**	16 → 6	20 → 2	43 → 17
`analytics.review_decision`	14 → **2**	14 → 2	35 → **1**	36 → 6
`llm.InvestmentAgent.ask`	12 → **6**	9 → 4	20 → 7	36 → 15
`mcp_client.parse_tool_result`	12 → **6**	8 → 5	9 → 3	17 → 7
`config.load_settings`	11 → **4**	10 → 4	6 → 3	34 → 12
`bot.cmd_stats`	10 → **4**	8 → 4	11 → 3	26 → 15

7.3. Модули

Модуль	MI (radon) до → после	Холстед V до → после	Ce до → после
`analytics`	46,25 → **51,32**	818,9 → 695,5	1 → 1
`bot`	31,46 → **38,49**	282,1 → 264,3	7 → 7
`market`	35,19 → **44,19**	705,3 → **204,0**	3 → 3
`config`	51,19 → 50,12	215,4 → 229,2	0 → 0

Модуль	MI (radon) до → после	Холстед V до → после	Ce до → после
`llm`	35,39 → 33,53	259,0 → 260,8	4 → 5
`mcp_client`	47,77 → 40,25	218,8 → 193,2	1 → 1
`models`	75,96 → 71,92	25,3 → 70,3	0 → 0
`__main__`	61,18 → 53,92	4,8 → 38,0	7 → 7

8. Выводы

1. Для Python построены все метрики из методички: индекс сопровождаемости (radon), цикломатическая сложность (radon, lizard, mccabe, pyscn), связанность классов CBO (pyscn), глубина наследования (pylint), строки исходного и исполняемого кода (radon raw). Дополнительно сняты метрики Холстеда, когнитивная сложность, LCOM4, связанность модулей Ca/Ce/I, дублирование и покрытие тестами. Аналогом правил CA1501/CA1502 служат flake8 C901 и проверки дизайна pylint с порогами в `.flake8` и `pyproject.toml`.
2. Метрики точно указали проблемные места. Шесть методов с CC > 10 при разборе оказались либо дублированием (`portfolio`, `review_decision`), либо смешением ответственностей (`candle_stats`, `ask`, `load_settings`). Независимые инструменты (radon, lizard, pyscn, pylint) указали на одни и те же методы. pylint W0212 нашёл скрытую связь через закрытое поле, которую CBO не учитывает.
3. После оптимизации нет ни одного нарушения правил сложности: максимальная CC снизилась с 30 до 7, все функции получили ранг A или B, максимальная когнитивная сложность — с 38 до 7, объём Холстеда модуля `market` уменьшился в 3,5 раза, покрытие выросло до 97 %.
4. **Суммарная сложность почти не изменилась (244 → 242), средний MI модулей тоже (48,3 → 48,2).** Декомпозиция не убирает существенную сложность задачи, а распределяет её по небольшим функциям, каждую из которых проще понять и протестировать. Реальное снижение есть только там, где устранено дублирование: у `market` MI вырос с 35 до 44.
5. У инструментов есть ограничения, которые нужно учитывать:
   - radon считает MI для модуля целиком, а не для каждого метода, как VS. Поэтому разбиение длинного метода на несколько функций почти не меняет MI модуля, а у модулей, которые стали длиннее (`mcp_client`, `llm`, `__main__`, `models`), он даже снизился. Формула radon учитывает и долю комментариев, так что MI можно «улучшить» комментариями, не меняя код;
   - разные инструменты по-разному определяют CC (учитывать ли `and`/`or`): у одного метода radon даёт 30, а mccabe — 8. Значения разных инструментов нельзя сравнивать напрямую, и порог правила нужно подбирать под конкретный инструмент;
   - LOC выросло на 48 строк из-за докстрингов и заголовков новых функций, хотя логических строк (LLOC) стало меньше. Для оценки объёма лучше подходят LLOC и SLOC;
   - pylint считает глубину наследования без `object`, а VS — с ним.

6. Выделение логики из обработчиков и точки входа дало побочный эффект: код стал тестируемым, и тест сразу нашёл реальный дефект (повторное подключение `Router`). Метрики стоит использовать как сигнал, где нужны ревью и тесты, а не как самоцель.